

8. Les constructeurs

8.1. Rôle du constructeur

Un **constructeur** est une méthode spéciale appelée automatiquement quand on crée un objet.

Son rôle principal est de **préparer l'objet dès sa création**.

À quoi sert-il ?

Il sert à :

- donner des valeurs de départ à l'objet ;
- recevoir des informations nécessaires ;
- éviter de créer un objet “vide” ou mal configuré.

Exemple simple

```
class User {  
    constructor(name, email) {  
        this.name = name;  
        this.email = email;  
    }  
}  
  
const user1 = new User("Awa", "awa@mail.com");
```

Ici, quand on écrit :

```
new User("Awa", "awa@mail.com")
```

le constructeur est appelé automatiquement.

Il remplit l'objet avec :

```
name = "Awa"
```

```
email = "awa@mail.com"
```

Image mentale

Une **classe** est comme un moule.

Le **constructeur** est l'étape où on met les informations nécessaires dans l'objet fabriqué.

À retenir

Le constructeur sert à **initialiser un objet au moment de sa création**. Sans constructeur, on risque d'avoir un objet incomplet ou mal préparé.

8.2. Initialisation d'un objet

Initialiser un objet, c'est lui donner ses **premières valeurs** au moment où il est créé.

En POO, cette initialisation se fait souvent dans le **constructeur**.

Exemple simple

```
class User {  
    constructor(name, email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

```
const user1 = new User("Awa", "awa@mail.com");
```

Ici, l'objet `user1` est initialisé avec :

```
name = "Awa"
```

```
email = "awa@mail.com"
```

Pourquoi c'est important ?

L'initialisation permet d'avoir un objet directement **prêt à être utilisé**.

Sans initialisation, on pourrait avoir un objet incomplet :

```
const user = {};
```

Ici, on ne sait pas encore son nom ni son email.

Image mentale

Créer un objet sans l'initialiser, c'est comme acheter un téléphone sans configuration.

L'initialisation, c'est le moment où on règle les informations de départ.

À retenir

Initialiser un objet, c'est **remplir ses données de départ dès sa création**.

8.3. Constructeur par défaut

Un **constructeur par défaut** est un constructeur utilisé quand on crée un objet **sans donner de valeurs personnalisées**.

Il permet de donner des **valeurs de départ automatiques**.

Exemple simple

```
class User {  
    constructor() {  
        this.name = "Invité";  
        this.email = "non défini";  
    }  
}
```

```
const user1 = new User();
```

Ici, on ne donne aucun paramètre :

```
new User()
```

Mais l'objet reçoit quand même des valeurs :

```
name = "Invité"
```

```
email = "non défini"
```

Pourquoi c'est utile ?

Le constructeur par défaut évite d'avoir un objet totalement vide ou inutilisable.

Image mentale 🧠

C'est comme créer un profil avec des informations de base déjà préparées.

À retenir

Un constructeur par défaut initialise un objet avec des **valeurs automatiques** quand aucune valeur n'est fournie.

8.4. Constructeur avec paramètres

Un **constructeur avec paramètres** permet de créer un objet en lui donnant des **valeurs personnalisées** dès le départ.

Les paramètres sont les informations qu'on envoie au constructeur.

Exemple simple

```
class User {  
    constructor(name, email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

```
const user1 = new User("Awa", "awa@mail.com");  
const user2 = new User("Moussa",  
"moussa@mail.com");
```

Ici, chaque objet reçoit ses propres valeurs :

```
user1.name = "Awa"
```

```
user2.name = "Moussa"
```

Pourquoi c'est utile ?

Cela permet de créer plusieurs objets à partir de la même classe, mais avec des données différentes.

Image mentale

La classe est comme un formulaire.

Le constructeur avec paramètres permet de remplir ce formulaire avec des informations différentes à chaque création.

À retenir

Un constructeur avec paramètres sert à **personnaliser un objet au moment où on le crée**.

8.5. Plusieurs constructeurs

Dans certains langages de POO, une classe peut avoir **plusieurs constructeurs**.

Cela permet de créer un objet de **différentes manières**.

Exemple d'idée

On pourrait créer un utilisateur :

```
new User()
```

```
new User("Awa")
```

```
new User("Awa", "awa@mail.com")
```

Chaque création donne plus ou moins d'informations.

Pourquoi c'est utile ?

Cela permet d'adapter la création d'un objet selon les données disponibles.

Par exemple :

```
class User {
```

```
    constructor(name = "Invité", email = "non  
défini") {
```

```
    this.name = name;

    this.email = email;
}
}
```

```
const user1 = new User();
const user2 = new User("Awa");
const user3 = new User("Awa", "awa@mail.com");
```

Ici, on simule plusieurs constructeurs avec des **valeurs par défaut**.

Attention

En JavaScript, une classe ne peut avoir qu'un seul `constructor`.

Mais on peut obtenir le même résultat avec :

- des paramètres optionnels ;
- des valeurs par défaut ;
- des méthodes de création.

À retenir

Plusieurs constructeurs servent à **créer un objet de plusieurs façons**, selon les informations qu'on possède au départ.

8.6. Surcharge de constructeurs

La **surcharge de constructeurs** signifie : avoir plusieurs constructeurs avec des **paramètres différents**.

Cela permet de créer un objet de plusieurs façons.

Exemple d'idée

Dans certains langages comme Java ou C# :

```
class User {
    User() {
```

```
// constructeur sans paramètre  
}  
  
User(String name) {  
    // constructeur avec un nom  
}  
  
User(String name, String email) {  
    // constructeur avec nom et email  
}  
}
```

On peut alors écrire :

```
new User();  
new User("Awa");  
new User("Awa", "awa@mail.com");
```

Pourquoi c'est utile ?

Parce qu'on n'a pas toujours toutes les informations au moment de créer l'objet.

La surcharge permet donc de créer un objet avec :

- aucune donnée ;
- quelques données ;
- toutes les données nécessaires.

Attention en JavaScript ⚠

JavaScript ne permet pas plusieurs `constructor` dans une même classe.

On utilise plutôt des **valeurs par défaut** :

```
class User {  
    constructor(name = "Invité", email = "non  
défini") {  
        this.name = name;  
        this.email = email;  
    }  
}
```

À retenir

La surcharge de constructeurs permet de **proposer plusieurs façons de créer un objet** selon les paramètres fournis.

8.7. Constructeur privé

Un **constructeur privé** est un constructeur qu'on ne peut pas appeler directement depuis l'extérieur de la classe.

Son rôle est de **contrôler la création des objets**.

Exemple d'idée

Dans certains langages comme Java ou C# :

```
class Database {  
    private Database() {  
        // constructeur privé  
    }  
}
```

Avec `private`, on empêche ceci :

```
new Database(); // interdit depuis l'extérieur
```

Pourquoi c'est utile ?

Un constructeur privé sert souvent à :

- empêcher la création libre d'objets ;

- forcer l'utilisation d'une méthode spéciale ;
- créer un seul objet partagé, comme dans le modèle **Singleton**.

Exemple simple avec Singleton

```
class Database {  
  
    private static Database instance = new  
    Database();  
  
    private Database() {}  
  
    public static Database getInstance() {  
        return instance;  
    }  
}
```

Ici, on ne crée pas la base avec `new Database()`.

On utilise :

```
Database db = Database.getInstance();
```

Image mentale 🧠

C'est comme une porte verrouillée :

on ne peut pas entrer directement, il faut passer par l'entrée autorisée.

À retenir

Un constructeur privé sert à **limiter et contrôler la création des objets**.

8.8. Constructeur protégé

Un **constructeur protégé** est un constructeur accessible seulement :

- dans la classe elle-même ;
- dans ses classes enfants.

Il empêche donc la création directe d'un objet depuis l'extérieur.

Exemple simple

```
class Animal {  
    protected Animal() {  
        System.out.println("Animal créé");  
    }  
}  
  
class Dog extends Animal {  
    public Dog() {  
        super();  
    }  
}
```

Ici, `Animal` possède un constructeur `protected`.

On ne peut pas faire directement :

```
Animal a = new Animal(); // interdit depuis  
l'extérieur
```

Mais une classe enfant peut l'utiliser :

```
Dog d = new Dog(); // autorisé
```

Pourquoi c'est utile ?

Un constructeur protégé sert à dire :

“Cette classe ne doit pas être créée directement, mais elle peut servir de base à d'autres classes.”

Image mentale 🧠

C'est comme une salle réservée à la famille :

les enfants de la classe peuvent entrer, mais pas n'importe qui.

À retenir

Un constructeur protégé sert à **contrôler la création d'objets tout en autorisant l'héritage**.

8.9. Validation dans un constructeur

La **validation dans un constructeur** consiste à vérifier les valeurs reçues avant de créer correctement l'objet.

Son but est d'éviter qu'un objet soit créé avec des données invalides.

Exemple simple

```
class User {  
    constructor(name, age) {  
        if (!name) {  
            throw new Error("Le nom est obligatoire");  
        }  
  
        if (age < 0) {  
            throw new Error("L'âge ne peut pas être négatif");  
        }  
  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
const user1 = new User("Awa", 25);
```

Ici, le constructeur vérifie que :

name existe

age n'est pas négatif

Pourquoi c'est utile ?

Cela permet de protéger l'objet dès sa création.

Par exemple, ceci est refusé :

```
const user2 = new User("", -5);
```

Parce que l'objet aurait des données incorrectes.

Image mentale 🧠

Le constructeur est comme un contrôleur à l'entrée :
il vérifie les informations avant de laisser l'objet exister.

À retenir

Valider dans un constructeur sert à créer uniquement des objets **corrects, fiables et utilisables**.

8.10. Erreurs fréquentes dans les constructeurs

Un constructeur sert à **préparer correctement un objet**.
Mais certaines erreurs peuvent rendre l'objet incomplet ou incorrect.

1. Oublier **this**

```
class User {  
    constructor(name) {  
        name = name; // erreur  
    }  
}
```

Ici, la valeur n'est pas stockée dans l'objet.

Correction :

```
class User {  
    constructor(name) {  
        this.name = name;
```

```
}  
  
}
```

`this.name` signifie : “la propriété `name` de cet objet”.

2. Ne pas valider les données

```
class User {  
    constructor(age) {  
        this.age = age;  
    }  
}
```

```
const user = new User(-5);
```

L’objet est créé avec un âge impossible.

Correction :

```
class User {  
    constructor(age) {  
        if (age < 0) {  
            throw new Error("Âge invalide");  
        }  
  
        this.age = age;  
    }  
}
```

3. Mettre trop de logique dans le constructeur

Un constructeur doit surtout **initialiser** l’objet.

Évite d'y mettre trop de traitements complexes : appels API, calculs lourds, logique métier longue, etc.

À retenir 🧠

Les erreurs fréquentes sont :

- oublier `this` ;
- ne pas vérifier les valeurs ;
- mettre trop de logique dans le constructeur.

Un bon constructeur crée un objet **simple, propre et valide**.

8.11. Constructeurs et invariants d'objet

Un **invariant d'objet** est une règle qui doit toujours rester vraie pour qu'un objet soit valide.

Le constructeur sert souvent à garantir ces règles **dès la création de l'objet**.

Exemple simple

```
class BankAccount {  
    constructor(balance) {  
        if (balance < 0) {  
            throw new Error("Le solde ne peut pas être négatif");  
        }  
  
        this.balance = balance;  
    }  
}
```



```
const account = new BankAccount(100);
```

Ici, l'invariant est :

```
balance >= 0
```

Cela signifie : un compte bancaire ne doit jamais avoir un solde de départ négatif.

Pourquoi c'est important ?

Sans invariant, on peut créer des objets incohérents :

```
const account = new BankAccount(-50); // interdit
```

Le constructeur protège donc l'objet contre les mauvaises valeurs.

Image mentale 🧠

Un invariant, c'est comme une règle de sécurité :
l'objet peut exister seulement si cette règle est respectée.

À retenir

Le constructeur ne sert pas seulement à remplir l'objet.

Il sert aussi à garantir que l'objet commence sa vie dans un état **valide et cohérent**.

8.12. Constructeurs et dépendances

Une **dépendance** est un objet dont une classe a besoin pour fonctionner.

Le constructeur peut recevoir ces dépendances au moment de créer l'objet.

Exemple simple

```
class EmailService {  
    send(message) {  
        console.log("Email envoyé :", message);  
    }  
}
```

```
class UserController {  
    constructor(emailService) {
```

```
        this.emailService = emailService;
    }

    welcomeUser() {
        this.emailService.send("Bienvenue !");
    }
}
```

```
const emailService = new EmailService();
const controller = new
UserController(emailService);
```

Ici, UserController dépend de EmailService.

Au lieu de créer EmailService directement à l'intérieur, on le donne au constructeur :

```
new UserController(emailService)
```

Pourquoi c'est utile ?

Cela rend le code :

- plus flexible ;
- plus facile à tester ;
- moins dépendant d'une seule implémentation.

Image mentale

C'est comme donner un outil à quelqu'un au moment où il commence son travail.

Le constructeur reçoit les outils nécessaires pour que l'objet fonctionne.

À retenir

Un constructeur peut recevoir les **dépendances nécessaires** pour créer un objet prêt à travailler correctement.